SYSTEM VALIDATION REPORT

Real-Time Chat Application Using Socket Programming

Mini Project Part-3, item 2(b)(ii)

Item	Details
Project	Real-Time Chat Application Using Socket Programming
Course	Software Engineering (UE23CS341A), PES University. Mini Project Part-3: Testing
Team	Kabir Raju G (Kabir7code) Keerthana K (KeerthanaK0304) Joshua (jjosh2005)
Repository	github.com/Kabir7code/Chat-application-Socket-Programming-
Test run	11 Nov 2026
Code tested	The complete Sprint 3 code: main on 11 Nov 2026 plus the Sprint 3 pull requests for #53, #54, #59, #60 and #61, which were being merged that day.

1. Purpose

This report checks that the finished system does what the SRS and the Sprint 3 backlog require. Each requirement is traced to the test cases and automated tests that validate it, and the result of each is recorded. It covers the integration (IT_xx) and system (ST_xx) test cases and the non-functional checks; unit-level results are in the Unit Test Report.

2. How the system was validated

- Every IT and ST case was run against a real server on localhost, with real sockets. Cases that describe a person using the app drive the actual terminal client (python -m chat.client) as a separate program, or the web UI in a real browser (Chromium through Playwright).
- Persistence cases start a server on a temporary database file, stop it, start a new server on the same file and check what is still there.
- Timing-sensitive cases were repeated: every case was run 4 times (ST_06, which leaves two clients idle for 10 minutes, once) with the same result each time. IT_08, the simultaneous-send ordering case, runs 200 rounds inside each run.
- Non-functional requirements were checked with the automated tests named below and a measured run with 20 users (section 5).
- Test run: 11 Nov 2026. Code: The complete Sprint 3 code: main on 11 Nov 2026 plus the Sprint 3 pull requests for #53, #54, #59, #60 and #61, which were being merged that day.

3. Summary

Cases	Run	Pass	Partial	Fail	Not executed
Integration (IT)	36	36	0	0	0
System (ST)	22	22	0	0	0
All cases incl. UT	88	86	2	0	0

Compared with Sprint 1 (42 Pass, 4 Partial, 5 Fail, 15 Not executed out of 66), every Sprint 1 failure except the terminal client's username check has been fixed, every group-chat case now runs and passes, and the 22 new Sprint 3 persistence cases all pass.

4. Requirement validation matrix

Req.	Requirement	Validated by	Result	Status
FR-001	Connect and register a unique username	UT_01, UT_02, UT_03, IT_01, IT_02, IT_03, ST_01, ST_02	6 Pass, 2 Partial. Terminal client does not pre-check names (UT_02/03, #29)	**Validated (minor finding)**
FR-002	Multiple users at the same time	UT_04, UT_05, UT_06, IT_04, IT_05, IT_06, ST_03, ST_04	8 Pass	**Validated**
FR-003	Real-time messaging	UT_07, UT_08, UT_09, IT_07, IT_08, IT_09, ST_05, ST_06	8 Pass	**Validated**
FR-004	Group chat management	UT_10, UT_11, UT_12, IT_10, IT_11, IT_12, IT_13, IT_14, ST_07, ST_08	10 Pass	**Validated**
FR-005	Private messaging	UT_13, UT_14, UT_15, IT_15, IT_16, IT_17, ST_09, ST_10	8 Pass	**Validated**
FR-006	Active users, group members, join/leave notices	UT_16, UT_17, UT_18, IT_18, IT_19, IT_20, ST_11, ST_12	8 Pass	**Validated**
FR-007	Leaving and disconnect handling	UT_19, UT_20, UT_21, IT_21, IT_22, IT_23, ST_13, ST_14	8 Pass	**Validated**
FR-008	Error handling on one client's socket	UT_22, UT_23, UT_24, IT_24, IT_25, IT_26, ST_15, ST_16	8 Pass	**Validated**
FR-009	Timestamps on every message	UT_07; test_timestamps.py	Every message has an ID and a server timestamp (UT_07, test_timestamps.py)	**Validated**
FR-010	Recent group message history	UT_28, IT_33, IT_34, IT_35	4 Pass	**Validated**
FR-011	Messages saved; chat history on login (Sprint 3)	UT_25, UT_26, IT_27, IT_28, ST_17, ST_21	6 Pass	**Validated**
FR-012	Private history and offline messages (Sprint 3)	UT_27, IT_29, IT_30, IT_31, IT_32, ST_18	6 Pass	**Validated**
FR-013	Groups survive a server restart (Sprint 3)	UT_28, IT_33, IT_34, IT_35	4 Pass	**Validated**
NFR-001	Delivery within 1 second	test_delivery_time.py; measured run (section 5)	20 users, 200 messages: slowest of 2000 deliveries took 61 ms (median 47 ms)	**Validated**
NFR-002	At least 20 users at once	test_load.py; ST_03, ST_04, ST_12; measured run	20 users in one group plus private messages: 1900/1900 group and 100/100 private deliveries, no crash	**Validated**
NFR-003	Reliable, correct delivery	test_protocol.py, test_ordering.py; IT_08, IT_27, ST_05	Framing keeps messages whole; same order for every user in 200 simultaneous-send runs	**Validated**
NFR-004	Usable interface; maintainable, configurable	UT_29, UT_30, IT_36, ST_19; test_web.py, test_logging.py, test_db_setting.py; UT_29, UT_30, ST_19, ST_21	Web UI (checked in a real browser) and terminal client work end to end; settings and logging tested	**Validated**
NFR-005	Registration gating and encryption	test_tls.py, test_robustness.py	Unregistered sockets are refused; a network capture shows no readable text with TLS on	**Validated**
NFR-006	Data survives a restart (Sprint 3)	ST_20, ST_21, ST_22	3 Pass	**Validated**
NFR-007	Database size kept in check (Sprint 3)	UT_29, UT_30, IT_36, ST_19	4 Pass	**Validated**

FR-011 to FR-013, NFR-006 and NFR-007 were added to the backlog for Sprint 3 (making the chat persistent, items #50 to #61) and are validated here; the SRS document itself still lists FR-001 to FR-010 and NFR-001 to NFR-005.

5. Measured performance (NFR-001, NFR-002)

20 users logged in, all joined one group, and over 10 rounds half of them sent a group message and the other half a private message (200 messages, saved to an SQLite file as in normal use). Delay is the time from sending to arriving at each receiver.

Measure	Group messages	Private messages
Deliveries (received / expected)	1900 / 1900	100 / 100

Measure	Group messages	Private messages
Median delay	47.4 ms	48.0 ms
95th percentile	56.4 ms	57.4 ms
Slowest	61.3 ms	60.9 ms

Every message arrived, nothing was lost or duplicated, and the slowest delivery was well under the 1-second limit.

6. Integration test results

ID	Description	Actual result	Result
IT_01	A new, unused username is accepted by the Server	Server replied 'registered' and 'rahul' is in the active-user list ['rahul'].	**Pass**
IT_02	A duplicate username is rejected by the Server	Second 'rahul' got error 'Username already taken.' (USERNAME_TAKEN); active-user list is still ['rahul'].	**Pass**
IT_03	Username uniqueness check behaves consistently for a repeated attempt	Usernames are case-sensitive (documented in the SAD): 'rahul' was accepted while 'Rahul' was online (registered); the same attempt repeated gave the same decision (registered).	**Pass**
IT_04	Two clients can connect at the same time and both are tracked	Both appear in the active list ['userA', 'userB'], each on its own socket and session.	**Pass**
IT_05	Activity on one client's session does not block or alter another's	While A sent 5 messages, B stayed connected and its own request was answered in 0 ms; B also received A's messages.	**Pass**
IT_06	Disconnecting one client leaves other sessions intact	Only userB was removed; list is ['userA', 'userC'] and A -> C messaging still works.	**Pass**
IT_07	Message sent by one client is received by another connected client	Client B received {from: clientA, text: 'Hi Client B'} immediately, with an ID and timestamp.	**Pass**
IT_08	Messages from multiple senders preserve arrival order	Three senders fired at the same instant, 200 runs: the two observers saw the same order every time, with correct senders (0 mismatches; was 4 in 300 in Sprint 1, fixed by #31).	**Pass**
IT_09	A message sent just before the recipient disconnects is not silently lost	The server never reports a false success: the first message was confirmed as delivered before B closed, and one sent after B left was reported as 'queued for later delivery' (FR-012), not dropped.	**Pass**
IT_10	A user can create a group and it appears in the group list	'StudyGroup' is in the server's group list with members ['clientA']; A got a 'created' update and is the admin.	**Pass**
IT_11	A second user can join an existing group	B joined; members are ['clientA', 'clientB'] and B's group message reached A.	**Pass**
IT_12	A message sent in a group reaches only that group's members	B received 'Meeting at 5pm' in StudyGroup; C received nothing (0 messages).	**Pass**
IT_13	A member who leaves a group stops receiving further messages from it	After B left, 'anyone still here?' did not reach B; members are now ['A'].	**Pass**
IT_14	Joining a group a user already belongs to is handled gracefully	Server answered 'You are already a member of 'StudyGroup'.' (ALREADY_MEMBER); membership not duplicated (['A']) and A stays connected.	**Pass**
IT_15	Private message is delivered only to the intended recipient	B received it; C received nothing (0 messages).	**Pass**
IT_16	Private message to a disconnected user is handled with a clear error	The message did not disappear silently: A was told '[19:57] [private to userB] hi (offline - delivered when they log in)'. Since FR-012 (#59) a user who has used the chat gets it on their next login instead of an error; a name that never logged in still gets...	**Pass**
IT_17	Private messaging does not interfere with group memberships	B got the private message and the group message on separate channels; C saw only 'hey team'.	**Pass**
IT_18	Other clients are notified when a new user joins	A and B both got 'userC joined the chat', and the online list (/users and the web page's list) now shows ['userA', 'userB', 'userC'].	**Pass**
IT_19	Other clients are notified when a user leaves	A and B both got 'userC left the chat'; the online list is now ['userA', 'userB'].	**Pass**
IT_20	Group member list updates for existing and joining members when someone joins	Both A and B received a group update listing ['A', 'B'], with no refresh needed.	**Pass**
IT_21	A client that leaves gracefully is immediately removed from the active-user list	A was removed from the active-user list in 20 ms, and B got 'A left the chat'.	**Pass**

ID	Description	Actual result	Result
IT_22	An abruptly killed client connection is eventually detected and cleaned up	Client process was force-killed (SIGKILL); the server removed A after 0.02 s (the OS closed the socket; TCP keepalive covers a lost network).	**Pass**
IT_23	Leaving one group does not disconnect the user from the server or other groups	A stays connected and is still in StudyGroup; only Squad membership was removed (Squad was deleted because it became empty).	**Pass**
IT_24	A malformed message from one client does not affect other connected clients	A got 'Message was not valid JSON.'; B stayed connected and kept receiving messages.	**Pass**
IT_25	Server recovers cleanly after a single client's abrupt socket error	Forced a TCP reset on A; the server cleaned A up and B -> C messaging was not interrupted.	**Pass**
IT_26	Client shows a clear message when it loses connection mid-session	After the server was stopped the client printed '[disconnected from server]' straight away.	**Pass**
IT_27	Saved messages are in the order users received them	Two users sent 30 messages each at once; the 60 saved messages are in exactly the order C saw them.	**Pass**
IT_28	A user who logs in is shown the recent Everyone chat	bob got the last 20 (m5 to m24), oldest first, right after 'registered'; the terminal shows them as '[19:57] (earlier) alice: m5'.	**Pass**
IT_29	/history shows the last 20 private messages with a user	/history bob returned the last 20 of 25 messages (p5 to p24), oldest first, both directions, each marked as history.	**Pass**
IT_30	A user cannot read other people's private conversations	carol asked for her history with alice and with bob: both empty. One side of the query is always the user asking, so alice-bob messages cannot be read.	**Pass**
IT_31	A private message to an offline user is kept and the sender is told	alice was told '[19:57] [private to bob] call me (offline - delivered when they log in)'; 1 message is waiting for bob.	**Pass**
IT_32	Waiting messages are delivered at login, in order, exactly once	On login bob got 'one', 'two', 'three' in order, shown as '[19:57] [private] alice (while you were away): one'; logging in again delivered nothing a second time.	**Pass**
IT_33	Groups, members and admin are restored after a server restart	After a restart Squad had members ['alice', 'bob'] and admin 'alice'; alice's group message reached bob without anyone rejoining.	**Pass**
IT_34	Group history is available after a server restart	After a restart carol joined Squad and was shown its last 10 messages (g2 to g11), marked earlier.	**Pass**
IT_35	A user who quits still leaves their groups (only a restart keeps them)	bob quit normally and was removed from Squad; the database holds {'Squad': ('alice', ['alice'])} (only a server restart keeps memberships).	**Pass**
IT_36	Only the newest messages of each chat are kept	With --keep 5 the Everyone chat kept ['e3', 'e4', 'e5', 'e6', 'e7'] and the alice-bob chat kept the newest 5; each chat is trimmed on its own.	**Pass**

7. System test results

ID	Description	Expected	Actual result	Result
ST_01	End-to-end successful connection and registration	Client reaches the main chat screen as 'priya_k'; Server's active-user list includes 'priya_k' within the expected connection time.	Registered in 0 ms; 'priya_k' is in the server's active-user list. The terminal client reaches the chat prompt ('Welcome, priya_k2! Online: priya_k').	**Pass**
ST_02	Client handles a Server that is unreachable at connection time	Client shows a clear connection-failure message (e.g. 'Unable to reach server') instead of hanging or crashing.	Client exited after 0.5 s with 'Could not connect to 127.0.0.1:<port> ([Errno 111] Connection refused)' instead of hanging.	**Pass**
ST_03	Server sustains multiple simultaneous connections under normal load	All 10 clients connect successfully, each session tracked independently, server stays responsive throughout.	All 10 clients registered in 0.01 s; the server tracks 10 sessions and a message from one reached the other 9.	**Pass**
ST_04	A new connection attempt does not disrupt existing connected users	The new client connects successfully; none of the 5 existing clients are disconnected, delayed, or lose in-flight messages.	New client joined during live chat; the 5 existing users stayed connected and kept receiving messages.	**Pass**
ST_05	Sustained real-time chat between two users over several exchanges	All 10 messages are delivered to the correct recipient, in order, with no manual refresh at any point.	All 10 alternating messages arrived at the right person, in order; slowest delivery 47.4 ms.	**Pass**
ST_06	Messaging keeps working correctly after a period of client inactivity	The message is still delivered normally, confirming the connection was not silently dropped during the idle period.	Both clients were left idle and connected for 600 seconds; 'back online' was then delivered normally.	**Pass**

ID	Description	Expected	Actual result	Result
ST_07	End-to-end group lifecycle: create, multiple joins, group messaging, and leave	Step 3's message reaches A and C (members at that time); step 5's message reaches only A and B, since C had left.	Step 3 reached A and C; after C left, step 5 reached only B (and A sent it).	**Pass**
ST_08	A user can belong to multiple groups at once without cross-delivery	Client A receives each message correctly attributed to its own group; no message appears under the wrong group.	A received each message under its own group: {'for study': 'StudyGroup', 'for squad': 'Squad'}.	**Pass**
ST_09	A user can hold simultaneous private conversations with two different users	Messages in each conversation are correctly attributed and never cross over to the other recipient.	B only got its 3 messages, C only got its 3, and A got 6 replies with the right senders.	**Pass**
ST_10	Private messaging remains functional alongside active group chat traffic	The private message reaches Client B correctly and is distinguishable from the concurrent group messages.	B got the private message while 20 group messages were arriving at A; A's screen shows it as '[19:57] [private to B] private during traffic', separate from '[Squad]' lines.	**Pass**
ST_11	Active-user list stays accurate through a sequence of joins and leaves	Final active-user list shows exactly {A, C, D}, matching the event sequence with no stale or missing entries.	Final active-user list on the server: ['A', 'C', 'D'].	**Pass**
ST_12	Presence notifications stay timely under moderate concurrent join/leave activity	All remaining clients' notifications and lists reflect the correct final state within expected latency bounds.	All 4 remaining clients received both join notices and the leave notice; slowest within 3 ms. Final list ['new1', 'new2', 'u0', 'u1', 'u2', 'u3'].	**Pass**
ST_13	Server remains stable when multiple clients disconnect in sequence	Server stays running and responsive throughout; final active-user list shows exactly the one remaining client.	After 2 /quit and 2 forced resets the server was still accepting connections; final list before the check was ['u4'].	**Pass**
ST_14	A user can reconnect after an unexpected disconnection	Reconnection succeeds and userA is re-added as a fresh session, with no leftover state blocking reuse of the username.	Reconnect with the same name succeeded ('registered') as a fresh session.	**Pass**
ST_15	Overall system stability under a burst of near-simultaneous client errors	Server stays running; the 2 affected sessions are cleaned up; the other 3 clients are completely unaffected.	One abrupt reset and one corrupt frame at the same moment: both sessions were cleaned up, the server kept running and the other 3 kept chatting (['u2', 'u3', 'u4']).	**Pass**
ST_16	End-to-end recovery: client reconnects and resumes normal use after an error-triggered disconnect	Client A reconnects successfully and all core messaging features (group, private) work normally.	userA was dropped for a bad frame, reconnected under the same name, rejoined Squad, and private, normal and group messages all worked.	**Pass**
ST_17	Everyone history survives a server restart	bob sees '(earlier) alice: before the restart' after logging in.	Server stopped and started again on the same chat.db; bob's terminal showed '[19:57] (earlier) alice: before the restart' after logging in.	**Pass**
ST_18	A message for an offline user survives a server restart	alice is told it will be delivered later; after the restart bob sees it marked '(while you were away)'.	alice saw '[19:57] [private to bob] see you after the restart (offline - delivered when they log in)'. After the server restarted, bob logged in and saw '[19:57] [private] alice (while you were away): see you after the restart'.	**Pass**
ST_19	The server creates the database where it is told; :memory: keeps nothing	The server logs where it saves messages and creates the file with all 5 tables; the :memory: server starts empty after a restart.	The server logged 'Saving messages to <tmp>/data/mychat.db' and created the file with tables ['group_members', 'groups', 'messages', 'offline_messages', 'users']. With --db :memory: a restart started empty (0 messages).	**Pass**
ST_20	All saved data survives two restarts in a row	dave sees 'hello all' and, on joining, 'hello squad'; alice's history with bob has 'hello bob'; Squad still has alice and bob.	After two restarts a new user saw '(earlier) alice: hello all', joining Squad showed 'hello squad', alice's /history bob returned ['hello bob'], and Squad still had alice and bob.	**Pass**
ST_21	The web UI shows saved history after a restart (#53)	Each chat shows its earlier messages, marked '(earlier)'; bob is back in Squad without rejoining.	After a server restart bob joined from the web page. Everyone: shown, Squad: shown, private chat with alice: shown.	**Pass**
ST_22	A database failure does not stop the chat	Both messages are still delivered, nobody is disconnected, and the error is logged.	With the database broken, messages were still delivered, nobody was disconnected, and the error was logged ('Could not save chat message from alice').	**Pass**

8. Defects and observations

#	Finding	Impact	Action
1	Terminal client does not check the username before sending (UT_02, UT_03). #29 was closed without changing chat/client.py.	Low: the server rejects the name with a clear message.	Reopen #29; add the check to register() in chat/client.py.
2	Shutdown race: ChatServer.stop() closes the database while a client thread may still be handling a message. Seen once while testing: a user logging in during shutdown got a server error and the error was logged.	Low: only during shutdown; that one message is not saved.	Close client sessions and wait for their threads before closing the store.
3	Web UI: after a server restart, a private chat appears in the chat list only when the other person is online or sends a new message; its saved history then loads (ST_21). There is no list of past conversations.	Low: history is not lost; /history <user> also works.	Possible future item: send the list of past private chats at login.
4	Behaviour change: a private message to an offline user who has used the chat is now kept and delivered later instead of returning an error (IT_16, FR-012).	Intended (Sprint 3).	Test case IT_16 updated; recorded in Deviations.md.

9. Conclusion

All 20 requirements were checked. 19 are fully validated and FR-001 is validated with one minor finding in the terminal client. The system meets the Part-3 exit criteria in the Test Plan: every case has a recorded result, the automated suite passes on Python 3.10 and 3.12, coverage is above target, and no open defect stops the demo flow.